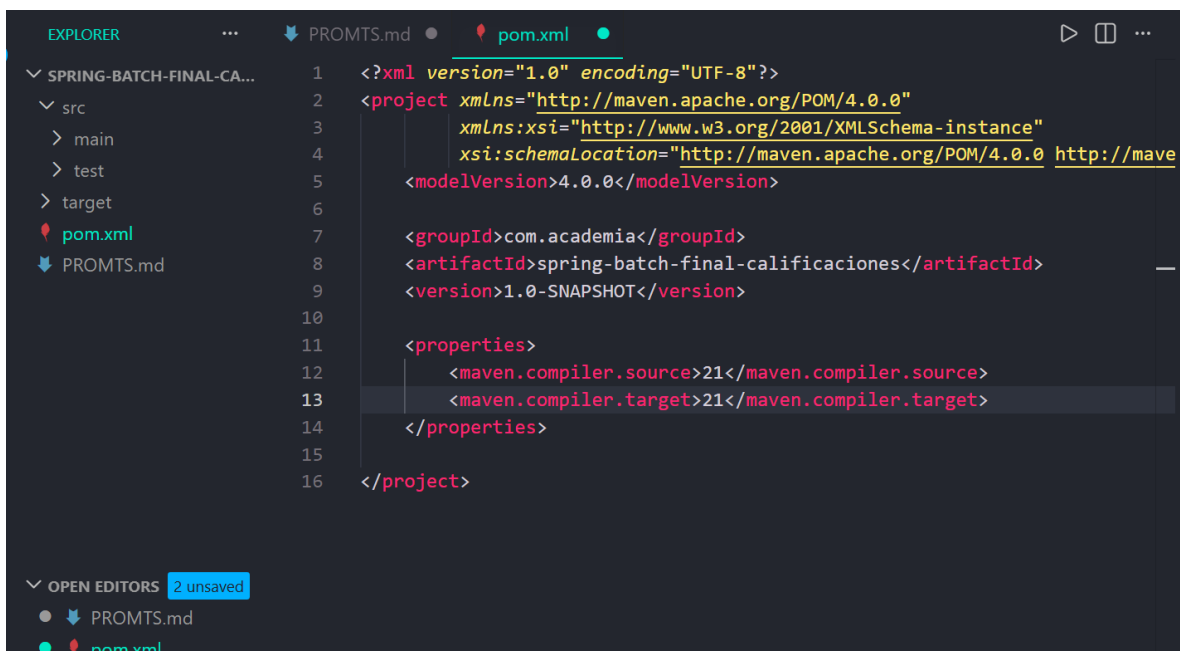


Creación de un Backend con IA Generativa y GitHub Copilot

A continuación se especifica el proceso de construcción y análisis de cada resultado obtenido por cada prompt

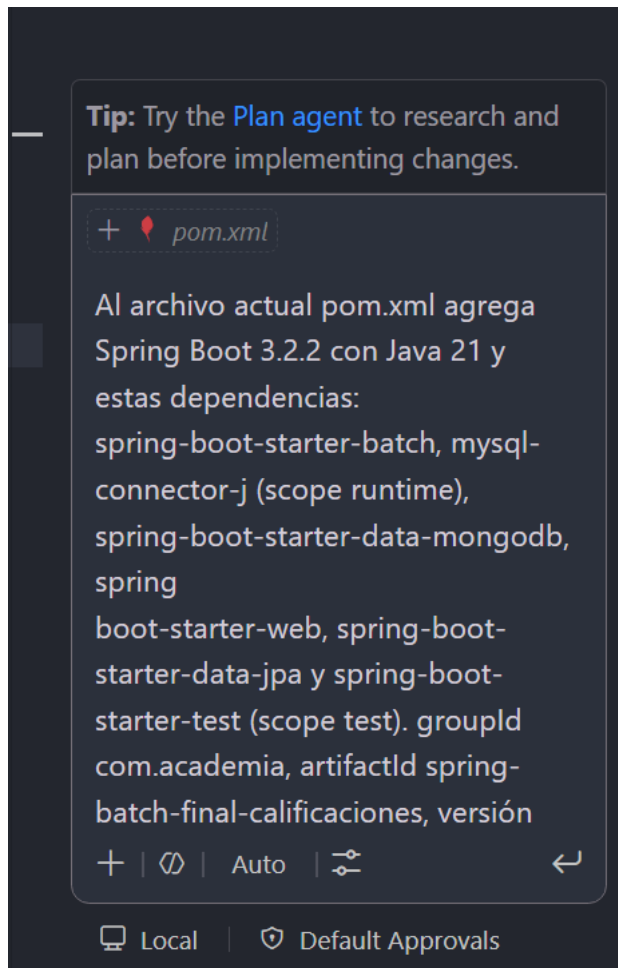
Estructura base del proyecto

Como primer paso se creo un proyecto en Maven básico como se muestra a continuación

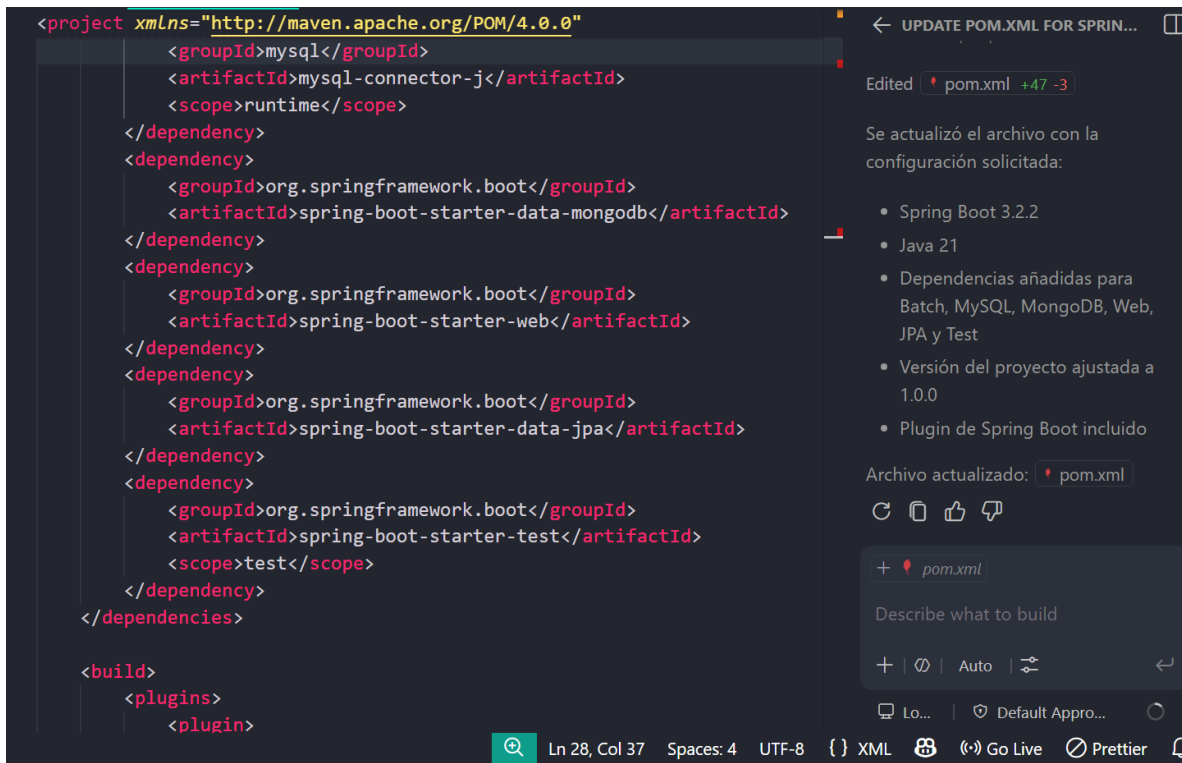


```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven
5         <modelVersion>4.0.0</modelVersion>
6
7         <groupId>com.academia</groupId>
8         <artifactId>spring-batch-final-calificaciones</artifactId>
9         <version>1.0-SNAPSHOT</version>
10
11        <properties>
12            <maven.compiler.source>21</maven.compiler.source>
13            <maven.compiler.target>21</maven.compiler.target>
14        </properties>
15
16    </project>
```

A continuación se hace la solicitud de las dependencias para el funcionamiento del programa



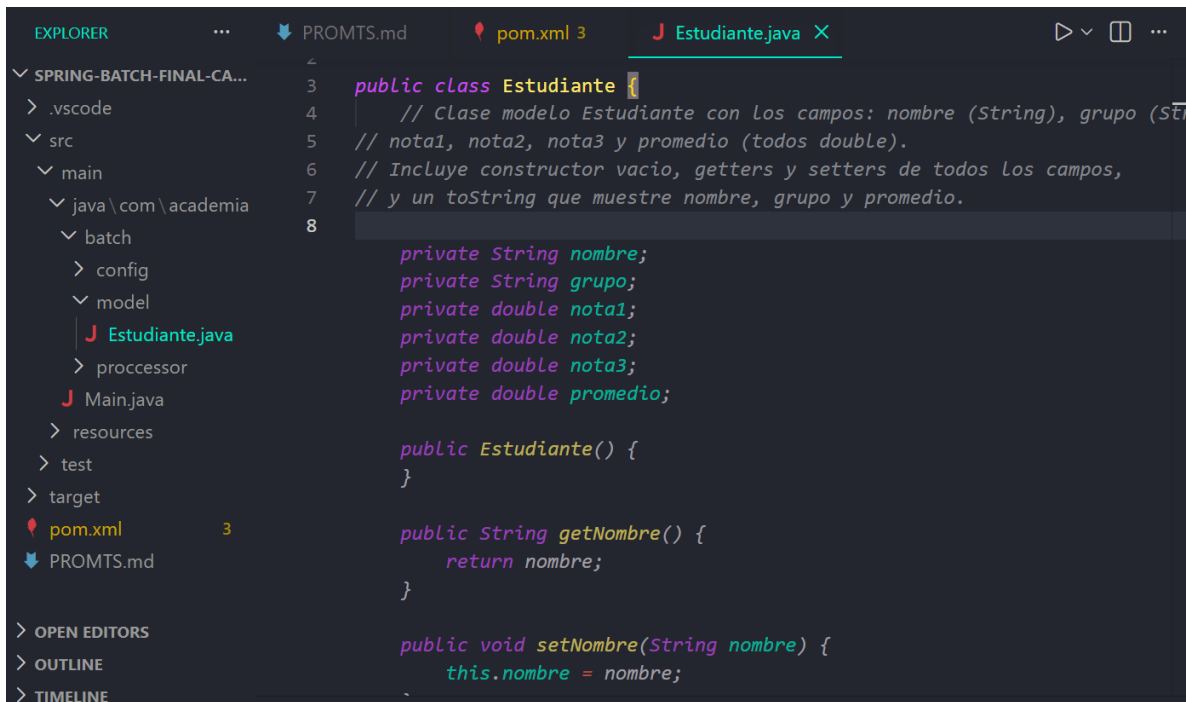
Verificación del archivo pom .xml



En este caso me doy cuenta que el groupId de la dependencia relacionada con mysql esta mal escrita por lo que procedo a corregirla

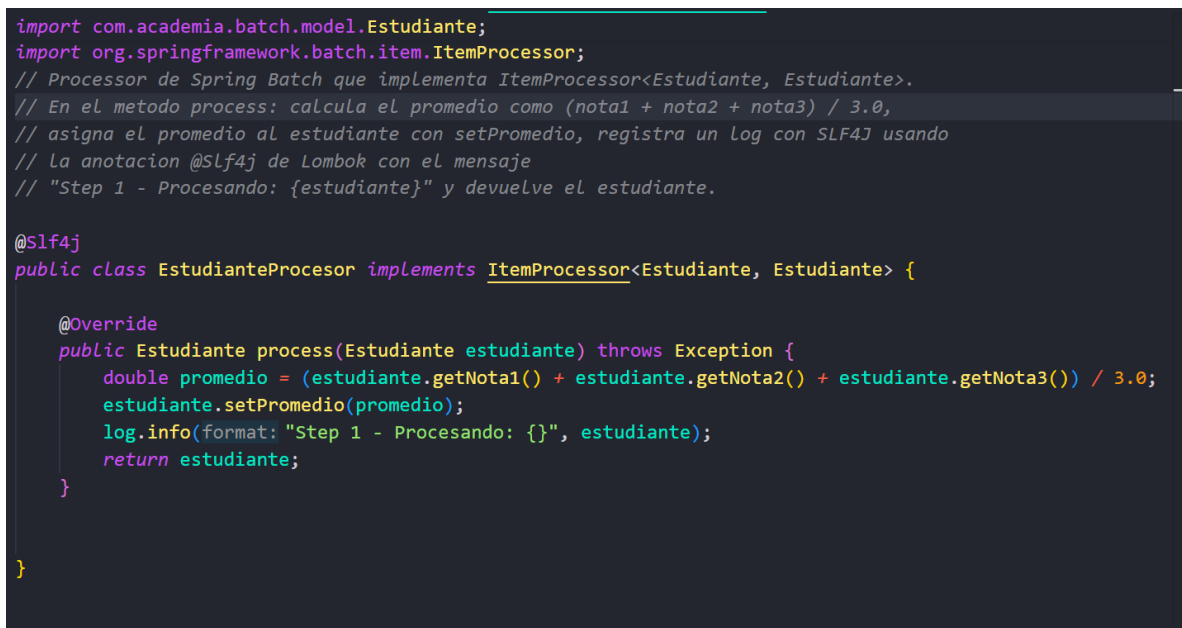


A continuación se procede a crear la clase POJO y se hace una descripción a modo de comentario de tal manera que la AI interpreta y sugiere el código



```
3 public class Estudiante {
4     // Clase modelo Estudiante con Los campos: nombre (String), grupo (String),
5     // nota1, nota2, nota3 y promedio (todos double).
6     // Incluye constructor vacio, getters y setters de todos Los campos,
7     // y un toString que muestre nombre, grupo y promedio.
8
9     private String nombre;
10    private String grupo;
11    private double nota1;
12    private double nota2;
13    private double nota3;
14    private double promedio;
15
16    public Estudiante() {
17    }
18
19    public String getNombre() {
20        return nombre;
21    }
22
23    public void setNombre(String nombre) {
24        this.nombre = nombre;
25    }
26
27    public String getGrupo() {
28        return grupo;
29    }
30
31    public void setGrupo(String grupo) {
32        this.grupo = grupo;
33    }
34
35    public double getNota1() {
36        return nota1;
37    }
38
39    public void setNota1(double nota1) {
40        this.nota1 = nota1;
41    }
42
43    public double getNota2() {
44        return nota2;
45    }
46
47    public void setNota2(double nota2) {
48        this.nota2 = nota2;
49    }
50
51    public double getNota3() {
52        return nota3;
53    }
54
55    public void setNota3(double nota3) {
56        this.nota3 = nota3;
57    }
58
59    public double getPromedio() {
60        return promedio;
61    }
62
63    public void setPromedio(double promedio) {
64        this.promedio = promedio;
65    }
66
67    public String toString() {
68        return "Estudiante [nombre=" + nombre + ", grupo=" + grupo + ", nota1=" + nota1 + ", nota2=" + nota2 + ", nota3=" + nota3 + ", promedio=" + promedio + "]";
69    }
70
71 }
```

En este punto se procede a agregar de forma manual la dependencia de loombok con la finalidad de hacer el registro de logs y ajustando el comentario se obtiene el siguiente resultado

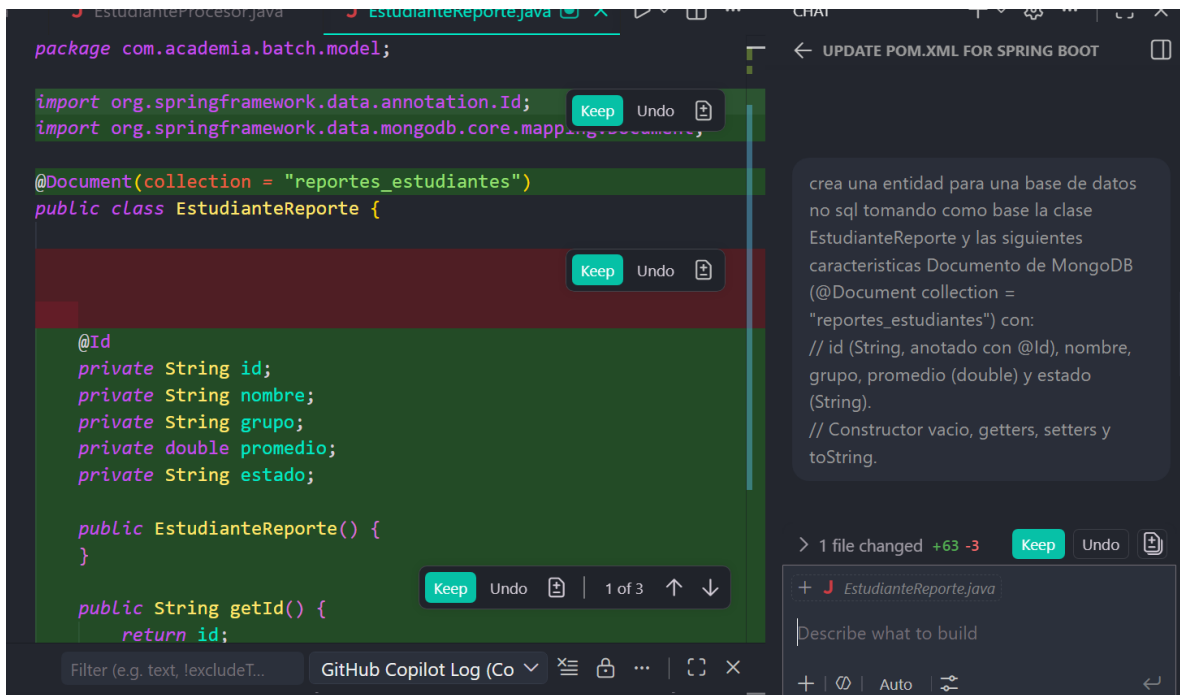


```
import com.academia.batch.model.Estudiante;
import org.springframework.batch.item.ItemProcessor;
// Processor de Spring Batch que implementa ItemProcessor<Estudiante, Estudiante>.
// En el metodo process: calcula el promedio como (nota1 + nota2 + nota3) / 3.0,
// asigna el promedio al estudiante con setPromedio, registra un Log con SLF4J usando
// la anotacion @Slf4j de Lombok con el mensaje
// "Step 1 - Procesando: {estudiante}" y devuelve el estudiante.

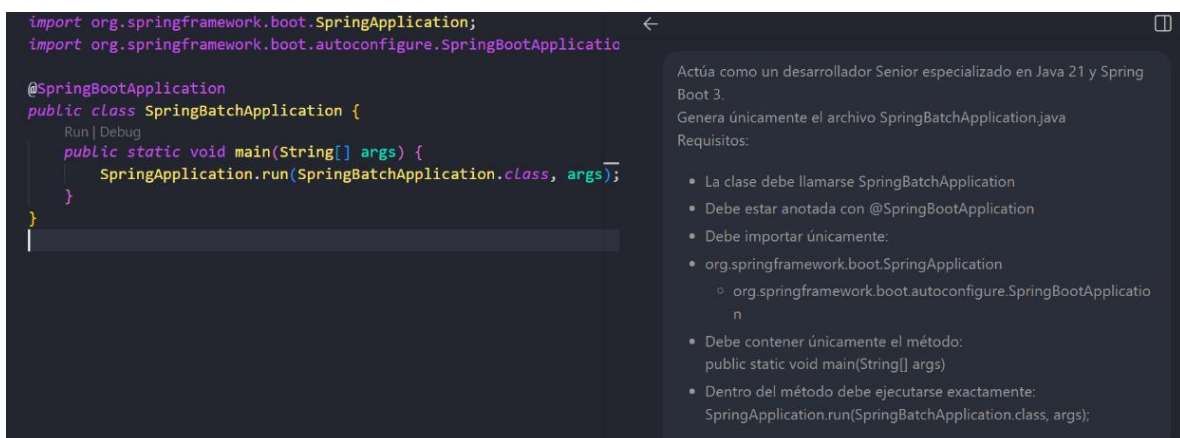
@Slf4j
public class EstudianteProcesor implements ItemProcessor<Estudiante, Estudiante> {

    @Override
    public Estudiante process(Estudiante estudiante) throws Exception {
        double promedio = (estudiante.getNota1() + estudiante.getNota2() + estudiante.getNota3()) / 3.0;
        estudiante.setPromedio(promedio);
        log.info(format: "Step 1 - Procesando: {}", estudiante);
        return estudiante;
    }
}
```

A continuación se procede a crear la clase EstudianteProcessor la cual es una entidad para una base de datos NoSQL



A continuación se muestra el prompt para el diseño de la clase principal en el cual se puede ver que se esta haciendo uso de las características principales de un buen prompt en el cual se le asigna a la AI contexto, objetivo y restricciones con la finalidad de hacer un mejor solicitud



Creacion del archivo .properties

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/academia
2 spring.datasource.username=alumno
3 spring.datasource.password=alumno123
4 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
5 spring.jpa.hibernate.ddl-auto=none
6 spring.jpa.show-sql=true
7 spring.jpa.properties.hibernate.format_sql=true
8 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect
9 spring.batch.jdbc.initialize-schema=always
10 spring.batch.jdbc.schema=classpath:org/springframework/batc
11 spring.batch.job.enabled=true
12
13 spring.data.mongodb.uri=mongodb://root:root123@localhost:27
14 spring.main.banner-mode=console
15 logging.pattern.console=%clr(%d{yyyy-MM-dd HH:mm:ss.SSS}){f
16 logging.level.org.springframework=INFO
17 logging.level.org.hibernate.SQL=DEBUG
18 logging.level.org.hibernate.type.descriptor.sql.BasicBinder
19
20
```

Genera únicamente el archivo application.properties para un proyecto Spring Boot 3.

Configuración de base de datos MySQL:

URL: jdbc:mysql://localhost:3306/academia
Usuario: alumno
Contraseña: alumno123

Incluye configuraciones compatibles con Spring Data JPA y Hibernate.

Configuración de Spring Batch:

Inicializar el esquema de Spring Batch en cada arranque.
Permitir configuración para controlar el

Filter (e.g. text, excludeT... Code X Copy ... [] X

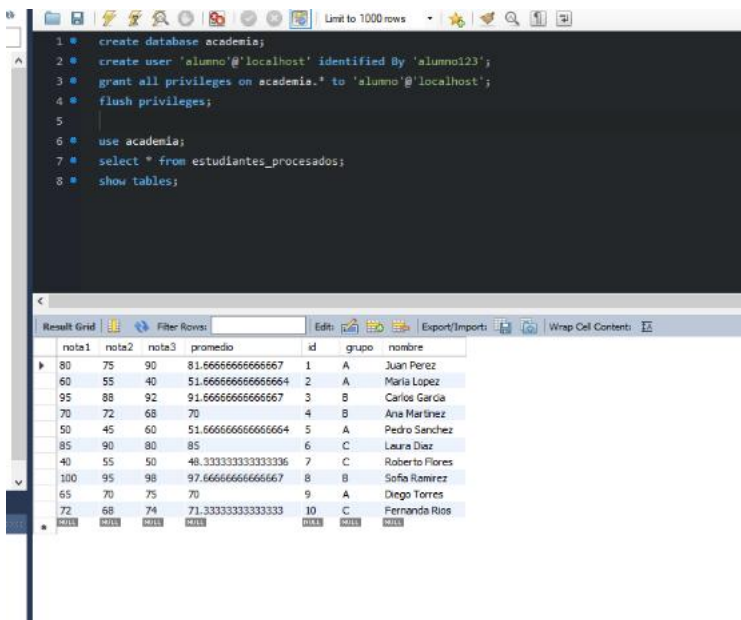
SpringBatchApplication.java:4: error: package org.springframework.boot.autoconfigure does not exist
import org.springframework.boot.autoconfigure.

+ application.properties

Describe what to build

+ | | Auto |

Después de ejecutar el comando `mvn spring-boot:run` se obtiene el siguiente resultado en la base de datos relacional



The screenshot shows a MySQL command-line interface with the following SQL commands and results:

```
1 create database academia;
2 create user 'alumno'@'localhost' identified by 'alumno123';
3 grant all privileges on academia.* to 'alumno'@'localhost';
4 flush privileges;
5
6 use academia;
7 select * from estudiantes_procesados;
8 show tables;
```

The results of the `select * from estudiantes_procesados;` query are displayed in a table with 10 columns: `nota1`, `nota2`, `nota3`, `promedio`, `id`, `grupo`, and `nombre`. The table contains 10 rows of data, including student names like Juan Perez, Maria Lopez, Carlos Garcia, Ana Martinez, Pedro Sanchez, Laura Diaz, Roberto Flores, Sofia Ramirez, Diego Torres, and Fernanda Rios.

nota1	nota2	nota3	promedio	id	grupo	nombre
80	75	90	81.66666666666667	1	A	Juan Perez
60	55	40	51.666666666666664	2	A	Maria Lopez
95	88	92	91.66666666666667	3	B	Carlos Garcia
70	72	68	70	4	B	Ana Martinez
50	45	60	51.666666666666664	5	A	Pedro Sanchez
85	90	80	85	6	C	Laura Diaz
40	55	50	48.333333333333336	7	C	Roberto Flores
100	95	98	97.66666666666667	8	B	Sofia Ramirez
65	70	75	70	9	A	Diego Torres
72	68	74	71.33333333333333	10	C	Fernanda Rios

Para el informe completo en la base de datos no relacional se tiene el siguiente resultado

```
test> show dbs
admin    100.00 KiB
config   12.00 KiB
local    72.00 KiB
test> db.getSiblingDB('academia').reportes_estudiantes.Find()
[
  {
    _id: ObjectId('6a496555ed949b704f4f014d'),
    nombre: 'Juan Perez',
    grupo: 'A',
    promedio: 81.66666666666667,
    estado: 'APROBADO',
    _class: 'com.academia.batch.modelo.EstudianteReporte'
  },
  {
    _id: ObjectId('6a496555ed949b704f4f014e'),
    nombre: 'Maria Lopez',
    grupo: 'A',
    promedio: 51.666666666666664,
    estado: 'REPROBADO',
    _class: 'com.academia.batch.modelo.EstudianteReporte'
  },
  {
    nombre: 'Pedro Sanchez',
    grupo: 'A',
    promedio: 51.666666666666664,
    nombre: 'Pedro Sanchez',
    grupo: 'A',
    promedio: 51.666666666666664,
    estado: 'REPROBADO',
    _class: 'com.academia.batch.modelo.EstudianteReporte'
  },
  {
    _id: ObjectId('6a496556ed949b704f4f0152'),
    nombre: 'Laura Diaz',
    grupo: 'C',
    promedio: 85,
    estado: 'APROBADO',
    _class: 'com.academia.batch.modelo.EstudianteReporte'
  },
  {
    _id: ObjectId('6a496556ed949b704f4f0153'),
    nombre: 'Roberto Flores',
    grupo: 'C',
    promedio: 48.333333333333336,
    estado: 'REPROBADO',
    _class: 'com.academia.batch.modelo.EstudianteReporte'
  },
  {
    _id: ObjectId('6a496556ed949b704f4f0154'),
    nombre: 'Sofia Ramirez',
    estado: 'REPROBADO',
    _class: 'com.academia.batch.modelo.EstudianteReporte'
  }
]
```

PARTE DOS API REST CON COPILOT

En este punto se usó un prompt más estructurado y con las características puntuales de tal manera que la AI no tenia espacio para poner algo incorrecto, el prompt completo se puede ver en el archivo PROMTS.md

The screenshot shows an IDE with a project structure on the left and a chat window on the right. The project structure includes:

- src
 - main
 - java\com\academia
 - batch
 - SpringBatchApplication.j...
 - Main.java
 - resources
 - application.properties
 - estudiantes.csv
 - test
 - target
 - .gitignore
 - pom.xml
 - Additional Views
 - README.md

The chat window displays a prompt in Spanish:

```
### 2. 'ReporteRepository'
```

The prompt text is:

```
Dentro del paquete batch crea un nuevo paquete llamado rest; dentro de este paquete quiero que crees los siguientes paquetes model, repository, service, view, dentro de model quiero que crees la clase EstudianteEntity la cual mapea a la tabla estudiantes_procesados. Los campos de esta tabla en la base de datos son id INT PRIMARY KEY AUTO INCREMENT, nombre VARCHAR(100) NOT NULL, grupo VARCHAR(10) NOT NULL, nota1 DECIMAL(5,2) NOT NULL, nota2 DECIMAL(5,2) NOT NULL, nota3 DECIMAL(5,2) NOT NULL, promedio DECIMAL(5,2) NOT NULL
```

Después de analizar que la inyección de dependencias y la lógica de negocio fuera la correcta se procede a verificar el código de estatus de dos solicitudes HTTP

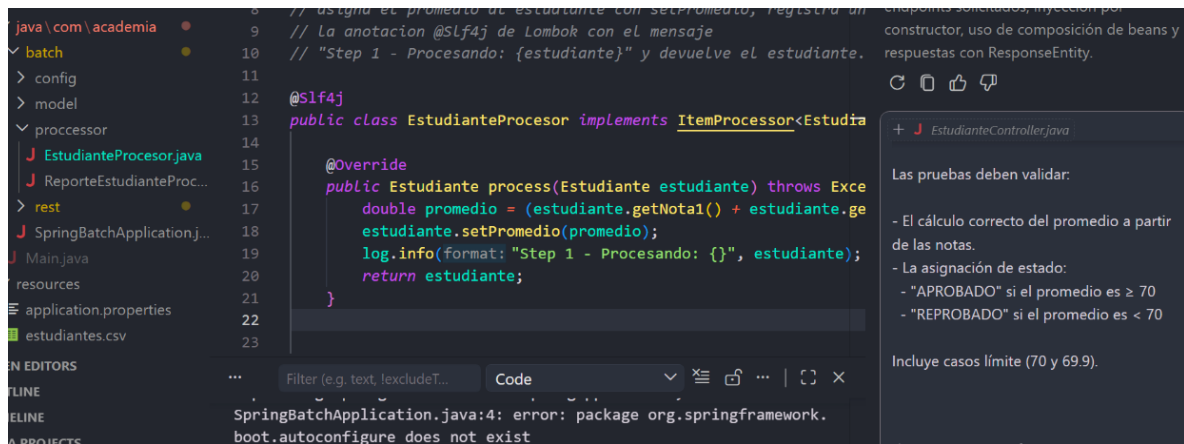
```
$ curl -i http://localhost:8080/api/estudiantes/9999
HTTP/1.1 404
Content-Length: 0
Date: Sat, 04 Jul 2026 21:30:23 GMT
```

```
jesus@DESKTOP-4F58DTA MINGW64 ~/Desktop/ProyectoAIFinal/BackendCopilotAI/spring-batch-final-calificaciones (main)
$ curl -i -X POST http://localhost:8080/api/estudiantes \-H "Content-Type: application/json" \-d '{"nombre":"jesus vazquez","grupo":"A","nota1":80,"nota2":85,"nota3":90,"promedio":85}'
HTTP/1.1 201
Content-Type: application/json
Transfer-Encoding: chunked
Date: Sat, 04 Jul 2026 21:35:52 GMT

{"id":22,"nombre":"jesus vazquez","grupo":"A","nota1":80,"nota2":85,"nota3":90,"promedio":85}
```

En las imágenes anteriores se puede ver que cuando se busca un id que no existe la api devuelve un 404 y cuando se crea con éxito un recurso, La api responde con un 201 lo cual corresponde con el estándar.

CREACIÓN DE CLASES TEST




```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.academia.batch.processor.EstudianteProcesorTest
16:11:59.980 [main] INFO com.academia.batch.processor.EstudianteProcesor -- Step 1 - Procesando: Es
tudiente{nombre='null', grupo='null', promedio=70.0}
16:12:00.041 [main] INFO com.academia.batch.processor.EstudianteProcesor -- Step 1 - Procesando: Es
tudiente{nombre='null', grupo='null', promedio=80.0}
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.429 s -- in com.academia.ba
ch.processor.EstudianteProcesorTest
[INFO] Running com.academia.batch.processor.ReporteEstudianteProcessorTest
16:12:00.066 [main] INFO com.academia.batch.processor.ReporteEstudianteProcessor -- Step 2 - Report
e: EstudianteReporte{id='null', nombre='Ana', grupo='A', promedio=70.0, estado='APROBADO'}
16:12:00.087 [main] INFO com.academia.batch.processor.ReporteEstudianteProcessor -- Step 2 - Report
e: EstudianteReporte{id='null', nombre='Luis', grupo='B', promedio=69.9, estado='REPROBADO'}
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in com.academia.ba
ch.processor.ReporteEstudianteProcessorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]

```

CONCLUSIONES

Al finalizar esta práctica pude comprender mejor cómo utilizar GitHub Copilot como una herramienta de apoyo durante el desarrollo de un proyecto con Spring Boot y Spring Batch. Aunque Copilot facilita mucho la generación de código, me di cuenta de que no siempre produce resultados correctos, por lo que fue necesario revisar cada sugerencia, corregir algunos errores y validar que todo funcionara antes de aceptarlo.

También reforcé conocimientos sobre Spring Batch, el procesamiento de archivos CSV, el uso de MySQL y MongoDB, la creación de una API REST y la elaboración de pruebas unitarias con JUnit y Mockito. Fue interesante comprobar que, aunque la IA ayuda a ahorrar tiempo, sigue siendo responsabilidad del programador entender el código que está generando y verificar que cumpla con los requisitos del proyecto.

En general, considero que esta práctica fue muy útil porque, además de desarrollar el backend solicitado, aprendí una mejor forma de trabajar con herramientas de inteligencia artificial, escribiendo mejores prompts y aprovechando sus funciones para generar código, explicarlo, documentarlo y corregir errores. Sin duda, GitHub Copilot puede aumentar la productividad, pero solo cuando se utiliza de manera crítica y responsable.